

BigWave E3 Formula Spec

작성일: 2026-05-26

문서 목적:

이 문서는 BigWave MVP v2의 E3 Trend Analyzer v1에서 사용되는 임시 산식과 각 함수의 책임을 정리한다.

중요:

- 현재 산식은 `TS\_v1\_temp`다.
- 정확도 검증이 끝난 최종 모델이 아니다.
- 산식은 나중에 쉽게 교체할 수 있도록 `feature\_builder\_v2.py`와 `ts\_scorer\_v2.py`에 고립되어 있다.

1. E3 산식 모듈 위치

```
bigwave_mvp_v2/analyzer_v2/  
    feature_builder_v2.py  
    ts_scorer_v2.py
```

`feature\_builder\_v2.py`:

- 날짜별 Trend Index 생성
- TS Score 계산에 필요한 feature 생성
- 정규화 유틸리티 제공

`ts\_scorer\_v2.py`:

- feature를 component score로 변환
- 최종 TS Score 계산
- 상태 라벨 부여

2. normalize_0_100(series)

위치:

```
feature_builder_v2.py
```

역할:

시계열 값을 0~100 범위로 정규화한다.

공식:

```
normalized = (value - min) / (max - min) * 100
```

예외 처리:

모든 값이 같고 0이 아니면 50
모든 값이 0이면 0
NaN은 0
최종 output은 0~100으로 clip

3. safe_log1p(series)

위치:

```
feature_builder_v2.py
```

역할:

조회수와 참여도처럼 값의 스케일이 큰 지표를 완화한다.

공식:

```
log_value = log(1 + value)
```

정책:

음수와 NaN은 0 처리 후 log1p 적용

4. build_trend_index(keyword_daily)

위치:

```
feature_builder_v2.py
```

역할:

날짜별 그래프용 Trend Index를 생성한다.

입력:

```
keyword_set_daily_metrics
```

중간 지표:

```
mention_index = normalize_0_100(weighted_mentions)
view_index = normalize_0_100(log1p(views))
engagement_index = normalize_0_100(log1p(engagements))
source_index = normalize_0_100(source_count)
```

최종 공식:

```
trend_index =
    mention_index * 0.50
    + view_index * 0.25
    + engagement_index * 0.15
    + source_index * 0.10
```

의미:

언급량: 50%
 조회수: 25%
 참여도: 15%
 채널 다양성: 10%

주의:

Trend Index는 날짜별 흐름 지표다.
 Trend Index는 TS Score와 다르다.

5. build_trend_features(keyword_daily, trend_index, weekly_partition)

위치:

```
feature_builder_v2.py
```

역할:

TS Score 계산에 필요한 기간 단위 feature를 만든다.

공식:

```
recent_mean = 최근 7일 trend_index 평균
previous_mean = 그 이전 7일 trend_index 평균
peak_trend_index = max(trend_index)
recent_slope = 최근 7일 마지막값 - 첫값
decline_from_peak = (peak_trend_index - recent_mean) / peak_trend_index
* 100
```

총량 feature:

```
total_mentions = sum(mentions)
total_weighted_mentions = sum(weighted_mentions)
total_views = sum(views)
total_engagements = sum(engagements)
```

반응 feature:

```
view_per_item = total_views / total_mentions
engagement_per_item = total_engagements / total_mentions
```

채널 다양성:

```
source_diversity = mean(source_count)
```

빈 데이터 또는 데이터 부족 시 모든 값은 안전하게 0으로 처리한다.

6. calculate_growth_score(features)

위치:

```
ts_scorer_v2.py
```

역할:

최근 흐름이 이전 구간보다 얼마나 성장했는지 계산한다.

공식:

```
growth_rate = (recent_mean - previous_mean) / previous_mean
growth_score = 50 + growth_rate * 50
```

예외:

```
previous_mean == 0 and recent_mean > 0 -> 70
previous_mean == 0 and recent_mean == 0 -> 0
```

최종 output은 0~100으로 clip한다.

7. calculate_reaction_score(features)

위치:

```
ts_scorer_v2.py
```

역할:

조회수와 참여도 기반 반응 강도를 계산한다.

Scalar 정규화:

```
view_component =
    log1p(view_per_item) / log1p(100000) * 100

engagement_component =
    log1p(engagement_per_item) / log1p(1000) * 100
```

최종 공식:

```
reaction_score =
    view_component * 0.60
    + engagement_component * 0.40
```

기준값:

```
view_per_item 상한 기준: 100000
engagement_per_item 상한 기준: 1000
```

이 기준값은 임시값이며 실제 사례 검증 후 조정한다.

8. calculate_saturation_score(features)

위치:

```
ts_scorer_v2.py
```

역할:

피크 대비 최근 흐름이 얼마나 유지되는지 계산한다.

공식:

```
saturation_score = recent_mean / peak_trend_index * 100
```

예외:

```
peak_trend_index == 0 -> 0
```

최종 output은 0~100으로 clip한다.

9. calculate_decline_risk(features)

위치:

```
ts_scorer_v2.py
```

역할:

피크 대비 하락과 최근 하락 기울기를 반영해 하락 위험을 계산한다.

기울기 패널티:

```
if recent_slope < 0:
    slope_penalty = min(abs(recent_slope) * 5, 100)
else:
    slope_penalty = 0
```

최종 공식:

```
decline_risk =
    decline_from_peak * 0.70
```

```
+ slope_penalty * 0.30
```

최종 output은 0~100으로 clip한다.

10. calculate_ts_score(component_scores)

위치:

```
ts_scorer_v2.py
```

역할:

component score들을 합쳐 최종 TS Score를 계산한다.

공식:

```
ts_score =  
    growth_score * 0.38  
    + reaction_score * 0.28  
    + saturation_score * 0.20  
    + (100 - decline_risk) * 0.14
```

의미:

성장성: 38%
반응 강도: 28%
피크 대비 유지력: 20%
하락 위험 방어: 14%

반환:

```
{  
    "ts_score": 74.1,  
    "growth_score": 93.0,  
    "reaction_score": 100.0,  
    "saturation_score": 21.9,  
    "decline_risk": 54.7,  
    "formula_version": "TS_v1_temp"  
}
```

11. assign_status_label(ts_score, decline_risk)

위치:

```
ts_scorer_v2.py
```

역할:

TS Score와 decline_risk를 바탕으로 상태 라벨을 부여한다.

기준:

```
decline_risk >= 65 and ts_score < 70 -> Declining Risk  
ts_score >= 80 -> Hot  
ts_score >= 65 -> Rising  
ts_score >= 45 -> Watch  
else -> Low
```

12. calculate_ts(features)

위치:

```
ts_scorer_v2.py
```

역할:

TS 계산 전체 wrapper다.

흐름:

```
features  
-> growth_score  
-> reaction_score  
-> saturation_score  
-> decline_risk  
-> ts_score  
-> status_label
```

최종 반환 예시:


```
{
  "ts_score": 74.1,
  "growth_score": 93.0,
  "reaction_score": 100.0,
  "saturation_score": 21.9,
  "decline_risk": 54.7,
  "formula_version": "TS_v1_temp",
  "status_label": "Rising"
}
```

13. 레거시 분석 흐름과의 연결

레거시 노트북의 핵심 흐름:

```
raw 수집
-> published_at datetime 변환
-> week/month 생성
-> keyword별 view_count 집계
-> content_supply 계산
-> view_per_video 계산
-> velocity / demand_pressure 계산
-> Z-score / 정규화
-> 그래프
```

E3 v1에서 유지한 흐름:

```
published_at 기준 시간축
daily/weekly partition
content_supply
view_per_item
engagement_per_item
정규화된 Trend Index
최근 구간 비교
피크 대비 하락
TS Score
dashboard_data.json
```

레거시의 시각화 코드는 직접 이식하지 않았다.

대신 프론트가 그래프를 그릴 수 있도록 `trend\_index.csv`와 `dashboard\_data.json`을 생성한다.

14. 교체 지점

산식 교체 시 주로 수정할 파일:

```
feature_builder_v2.py  
ts_scorer_v2.py
```

프론트 응답 구조 수정 시:

```
dashboard_packager_v2.py
```

파일 저장 정책 수정 시:

```
analyzer_router_v2.py
```